어텐션과 트랜스포머

Q·K·V와 어텐션의 역사 · 스케일드 닷-프로덕트 어텐션과 셀프-어텐션 · 멀티헤드와 포지셔널 인코딩

Machine Learning 1 · 12주차

한국과학영재학교 (KSA)

Week 12

이번 주차 개요

- **12.1 어텐션과 Q·K·V** — RNN 병목, 2014년 기계번역, Q·K·V 손계산
- **12.2 스케일드 닷-프로덕트** — 4단계 손계산, $\sqrt{d_k}$ 스케일링
- **12.3 멀티헤드와 PE** — 관점 h배, 시계 PE, causal mask

이 주차를 마치면

Q·K·V 세 역할, softmax$(QK^T/\sqrt{d_k})V$ 의 4단계, 멀티헤드($d_k = d_{model}/h$), 포지셔널 인코딩까지 — 다음 주차 LLM 기계의 돌아가는 방식을 이미 알게 된다.

2017년 “Attention Is All You Need” — 제목의 “All”은 무엇이 전부였던가?

12.1 어텐션 아이디어와 Query·Key·Value

RNN의 근본 문제: 순차성과 압축 병목

- 문제 (1): GPU는 **병렬 연산**에 특화됐는데, **순차 구조는 병렬성을 활용하지 못해** 느리다.
- 문제 (2): 먼 과거의 정보가 여러 단계를 거치며 **흐려진다** (Week 11의 그래디언트 소실과 정확히 같은 문제).

덜 눈에 띄는 세 번째 한계: 압축 병목

RNN의 t번째 은닉 상태는 $1 \sim t$번 입력만 볼 수 있다 — **미래** 정보는 순전파 단계에서 현재에 아예 도달하지 못한다. 기계번역처럼 "원문을 전부 읽은 뒤 번역을 시작해야 하는" 작업에서는, 원문을 처음부터 끝까지 읽은 뒤 **마지막 은닉 상태 하나**에 원문 전체의 의미를 압축해 담을 수밖에 없다 (Sutskever et al., 2014 인코더-디코더 구조가 이 형태를 강제). 문장이 길수록 한 벡터에 더 많은 정보를 구겨 넣어야 해 그만큼 잘 안 된다 — BPTT 그래디언트 소실과 같은 "길이에 대한 취약성"의 또 다른 모습.

어텐션(attention)의 도입

어텐션의 직관: 모든 단어를 한 번에

"지금 이 단어를 이해하려면, 문장의 **다른 모든 단어를 한 번에** 보고, 그중 관련 있는 단어에 더 집중(attend)한다."

RNN과의 차원이 달라지는 수식

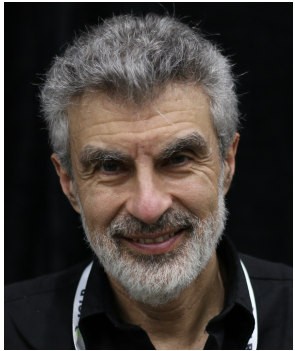
어텐션은 현재 단어의 새 표현을 만들 때 n개 단어 각각에 **가중치 하나씩, 총 n개**를 계산 — "단어 위의 분포" 를 만든다. 이 n차원 벡터를 **어텐션 가중치**라 부른다. "그것은"의 가중치는 (0.56, 0.01, 0.01, 0.42) — "동물"에 56%, 자기 자신에 42% (숫자는 12.2절 실습).

RNN이 주던 것: "과거의 **압축 요약(한 벡터)**". 어텐션이 주는 것: **"관련 단어들의 내용을 적절한 비율로 섞은 것"**.

역사: 어텐션의 3년

연도	논문
2014	Bahdanau · Cho · Bengio, Neural Machine Translation by Jointly Learning to Al
2015	Luong et al.
2017	Vaswani et al., Attention Is All You Need

어텐션의 기원은 RNN의 패치



Yoshua Bengio
2014 Bahdanau · Cho · **Bengio**
NMT 어텐션 공동제안
(2018 ACM Turing Award)



Llion Jones
2017 Attention Is All You Need
8인 공저자
(현 Sakana AI 공동창업자)



Aidan Gomez
같은 논문 8인 공저자
(현 Cohere CEO)



Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani[*] Google Brain avaswani@google.com

Noam Shazeer[*] Google Brain noam@google.com

Niki Parmar[*] Google Research nikip@google.com

Jakob Uszkoreit[*] Google Research usz@google.com

Llion Jones[*] Google Research llion@google.com

Aidan N. Gomez[* †] University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser[*] Google Brain lukaszkaiser@google.com

Illia Polosukhin[* ‡] illia.polosukhin@gmail.com

arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

[*]Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Vaswani et al., Attention Is All You Need (2017), NeurIPS 2017 — 8인 공저(Google Brain/Research), 원문 arXiv 1페이지.

Query, Key, Value — 세 가지 역할

각 단어(임베딩 벡터 x)는 세 가지 벡터로 변환된다:

- **Query (Q)**: “내가 지금 무엇을 **찾고** 있는가”
- **Key (K)**: “나는 어떤 정보를 **갖고** 있는가”
- **Value (V)**: “실제로 **전달**할 내용”

세 벡터 모두 같은 입력 임베딩 x에 **서로 다른** 학습 가능한 가중치 행렬을 곱해 얻는다:

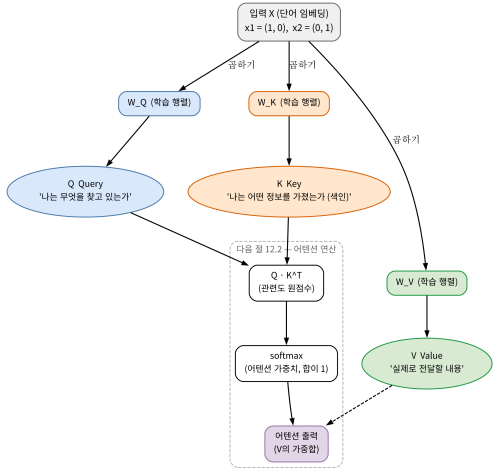
$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

도서관 비유와 전체 흐름

도서관 비유

- **Query** = 내가 찾는 책의 **주제**(질문)
- **Key** = 각 책의 **색인 카드**에 적힌 요약
- **Value** = **실제 책의 내용**

Query를 모든 책의 Key와 비교 → 가장 관련 있는 책들을 찾는다.
그 관련도만큼 실제 내용(Value)을 **섞어서 가져오는** 것이 다음 절의 어텐션 연산이다.



같은 입력 임베딩 x_1, x_2에 서로 다른 세 행렬 W_Q, W_K, W_V 를 곱해 Q·K·V를 만들고, QK^T 점수 → softmax → V 가중합으로 어텐션 출력을 만든다.

손으로 한 번: 같은 입력, 서로 다른 Q·K·V

단어 2개, 임베딩 $x_1 = (1, 0),\ x_2 = (0, 1)$ (즉 $X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$). 서로 다른 세 변환 행렬:

$$W_Q = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}, \quad W_V = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

$Q = XW_Q, K = XW_K, V = XW_V$ 를 계산하면:

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad K = \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}, \quad V = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

핵심 관찰

같은 입력 X 에서 출발했는데도 Q, K, V 가 **서로 완전히 다른 값**으로 나온다 — 세 행렬이 각각 **독립적인 학습 파라미터**를 갖고 있기 때문이다.

세 행렬의 “역할”은 학습이 만든다

2단어 예의 세 행렬이 각각 하는 일:

행렬	효과	역할
W_Q	두 번째 차원을 2배로 확대	“찾는 방향”을 세우
W_K	모든 차원을 절반으로 축소	“색인”의 스케일을
W_V	두 차원을 서로 섞음 (비대각 원소 $\neq 0$)	전달할 **내용**을 다른

역할은 학습을 통해 만들어진다

왜 “관련성”은 내적으로 재는가

- **내적은 방향 일치도에 비례**
- 직각이면 0, 반대 방향이면 음수.
- “찾는 정보의 방향” = Query, “가진 정보의 방향” = Key 이면, **Q · K는 찾는 것과 가진 것의 방향이 일치할수록 큰 숫자** — “관련성”의 정의 그 자체.

전체 파이프라인 미리보기: 3단계

1. 모든 단어 쌍의 **원점수(raw score)** QK^T 계산
2. 각 행에 **softmax**를 적용해 분포(어텐션 가중치) 생성
3. 그 가중치로 V 의 행을 **가중합** $\rightarrow$ 출력

$$QK^T = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix} = \begin{pmatrix} 0.5 & 0 \\ 0 & 1 \end{pmatrix}$$

계산 이어가기: 스케일링 · softmax · 가중합

$\sqrt{d_k} = \sqrt{2}$ 로 나눈 뒤 각 행에 softmax:

최종 출력 ($V_1 = (2,1), V_2 = (1,2)$):

- 단어 1: $0.587\,V_1 + 0.413\,V_2 = (\mathbf{1.587},\ \mathbf{1.413})$
- 단어 2: $0.330\,V_1 + 0.670\,V_2 = (\mathbf{1.330},\ \mathbf{1.670})$

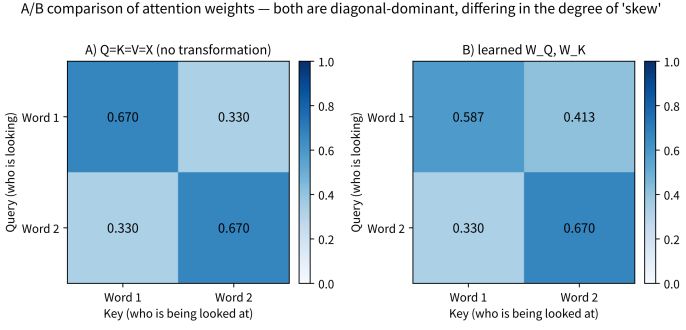
두 가지 관찰

(1) **단어 1의 출력이 V_1 쪽에, 단어 2의 출력이 V_2 쪽에 기울어 있다** (0.59, 0.67) — "관련 단어의 내용을 관련도만큼 섞는다"가 숫자로 성립. (2) 출력이 순수한 복사 (정답이면 (2,1) · (1,2))는 **아니다** — 다른 단어의 내용이 조금 섞여 있다.

A/B 비교: 변환 없이 Q=K=V=X로 쓰면

Q=K=V=X 어텐션 계산 결과

경우	어텐션 가중치 (단어 1 행)	단어 1의 출력
$Q = K = V = X$ (변환 없음)	(0.670, 0.330)	(0.670, 0.330)
학습된 W_Q, W_K, W_V	(0.587, 0.413)	(1.587, 1.413)



변환이 없으면 두 가지가 깨진다

1. W_Q, W_K, W_V**는 학습의 유일한 자리**

핵심

기울기가 실제로 흘러 W_Q, W_K, W_V 가 갱신되는 부분이 곧 모델이 “어떤 관계를 보게 될지”를 결정하는 부분이다 — 이 세 행렬의 존재가 어텐션 메커니즘의 **핵심**.

자주 묻는 질문 1: 왜 세 가지로 나뉘나

Q·K·V는 왜 같은 입력에서 서로 다른 세 가지로 나뉘나요? 입력 자체를 비교하면 안 되나요?

- $Q = K = V = X$ (변환 없이 원본 입력을 그대로 씀)라면 “이 단어가 자기 자신과 얼마나 비슷한가”만 재게 되어 **표현력이 크게 제한**된다.
- 서로 다른 학습 가능한 변환 W_Q, W_K, W_V 를 두면, “무엇을 찾을지”와 “무엇을 제공할지”를 모델이 **데이터로부터 독립적으로 학습**할 수 있다.
- 예: 대명사는 “나를 가리키는 명사를 찾는다” 방향으로 W_Q 가, 명사는 “대명사가 찾는 대상이 될 수 있다” 방향으로 W_K 가 학습된다.

보충: “내적(유사도)으로 관련을 재고, 학습된 변환을 거친 뒤 비교한다”는 패턴은 Week 5.3 커널 함수 $K(x, x')$ 과 정신적으로 닮아 있다.

자주 묻는 질문 2: Q와 K가 중복되는 것 아닌가

Q, K가 같은 X에서 나오는데, 출력이 "X를 하나의 행렬로 변환한 것"과 같지는 않나요?

- **중복이 아니다.** 점수 QK^T 는 **두 행렬의 곱**으로 결정된다 — 같은 X 여도 W_Q 와 W_K 가 무엇이냐에 따라 "어떤 단어 쌍이 만나고 어떤 쌍이 안 만나는지"가 완전히 달라진다.
- 같은 X 를 "질문 카드·색인 카드·내용 카드" 세 장으로 찍어 내는 것과 같다 — 질문·색인 카드는 **비교 단계**에서만, 내용 카드는 **전달 단계**에서만 쓰인다.
- $Q = K$ 로 고정하면 QK^T 가 **대칭**이 되어 "내가 너를 찾는 정도" = "너가 나를 찾는 정도"가 되어 **비대칭적 관계**를 표현할 수 없다.

예: "대명사는 명사를 찾지만, 명사는 대명사를 찾지 않는다" — 실제 모델에서 어텐션의 **방향성**(어떤 헤드는 뒤쪽, 어떤 헤드는 앞쪽 단어를 본다)은 바로 이 **비대칭성**에서 나온다.

그렇다면 왜 편향(bias)이 아니라 행렬 곱인가?

- 편향은 모든 단어에 **같은** 벡터를 더하는 것 — "단어 A는 대명사고 단어 B는 명사"를 구분할 수 없고, 단어 사이의 **상대적 기하학**(각도, 방향)이 그대로다.
- 내적 점수는 바로 그 상대적 기하학으로 결정된다 — 점수의 의미를 바꾸려면 방향 자체를 돌리거나 늘리거나 섞을 수 있어야 한다(선형 변환). 선형만으로는 공간을 "접을" 수 없다 (Week 9.2) — 어텐션에서는 그 "접는 역할"을 **점수 비교**가 대신 맡는다.

자주 묻는 질문 4: 실제 모델에서 얼마나 큰가

2017년 원논문(Vaswani et al.)의 스케일:

- 모델 차원 $d_{model} = 512 \Rightarrow W_Q, W_K, W_V$ 는 각각 **512**×**512** 행렬 (각각 약 **26만**개 파라미터).
- 감각 비교: Week 6 당뇨 회귀 문제가 특징 10개인 것과 비교하면 스케일을 잡기 좋다.
- 12.3절 멀티헤드에서는 이 512차원을 $h = 8$ 개 헤드에 나눠 쓰므로 각 헤드 행렬은 **64**×**64**가 된다.

세 행렬 모두 **학습으로 결정**된다 — 사람이 "이 행렬은 대명사 담당"이라고 설계하지 않는다.

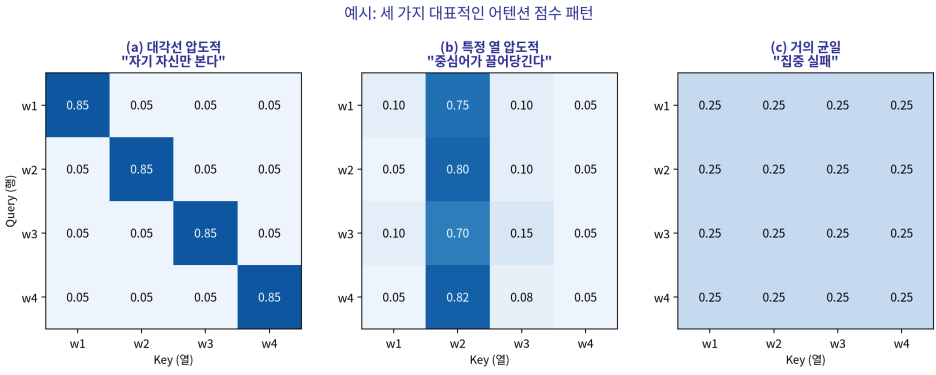
12.3절에서 보듯, 헤드들이 각자 다른 역할로 **스스로 분화** 한다는 것을 숫자와 관측으로 확인한다.

자주 하는 실수: “검색” 단계를 건너뛰기

- 점수(QK^T)가 V의 혼합을 결정
- V는 무엇을, 점수는 얼마나 mix할지

- 대각선 압도적: 자기 자신만 보는 패턴
- 특정 열 압도적: 중심어가 끌어당김
- 거의 균일: 집중 실패

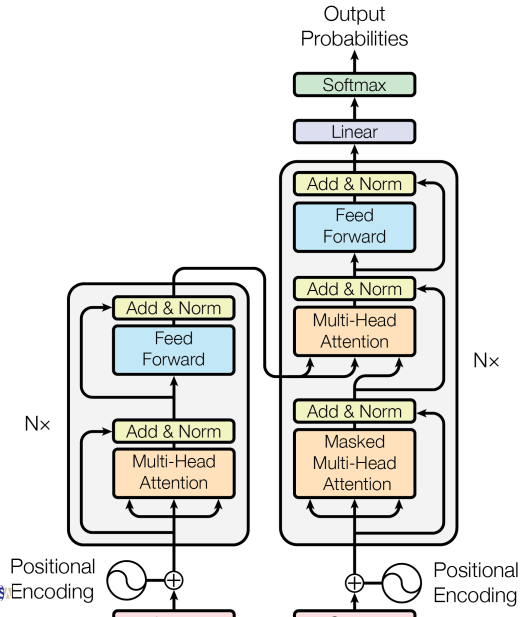
그림으로: 세 가지 어텐션 점수 패턴



대각선 압도(자기 자신만 본다) · 특정 열 압도(중심어가 끌어당긴다) · 거의 균일(집중 실패) — 어텐션 점수 행렬 QK^T(softmax 후)을 볼 때 가장 먼저 확인할 세 가지 패턴.

12.2 스케일드 닷-프로덕트 어텐션과 셀프-어텐션

트랜스포머 원논문의 핵심 연산



Scaled dot-product attention은 Transformer 원논문의 핵심 연산. 한 단어의 Query가 모든 단어의 Key와 얼마나 "관련 있는가"를 내적(dot product)으로 잰다 (Bahdanau et al., 2014; Luong et al., 2015):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k = Key 벡터의 **차원** (12.2 말미 실수 섹션 참고).

4단계로 뜯어보기

1. QK^T: 단어 쌍 Query-Key 내적
2. softmax: 확률분포 변환
3. V 가중합: 관련도별 내용 혼합

코드로 4단계

(실습)

```python
import math

def softmax(row):
    m = max(row)
    exps = [math.exp(v - m) for v in row]
    total = sum(exps)
    return [e / total for e in exps]

def attention(Q, K, V, d_k):
    scores = [[sum(Q[i][t] * K[j][t] for t in range(d_k))
               / math.sqrt(d_k) for j in range(len(K))]
              for i in range(len(Q))]          # QK^T / sqrt(d_k)
    weights = [softmax(row) for row in scores] # softmax
    output = [[sum(weights[i][j] * V[j][t] for j in range(len(V)))
               for t in range(len(V[0]))]
              for i in range(len(Q))]          # V 가중합
    return output, weights
```

코드 읽기: 4단계가 그대로 번역

- `scores[i][j]` = $Q[i] \cdot K[j] / \sqrt{d_k}$ — QK^T 의 (i, j) 성분. `scores` 전체가 $n \times n$ **원점수 행렬** 자체.
- `weights[i]` = `scores[i]`에 softmax를 적용한 결과 — i번째 단어(행)가 j번째 단어들(열)에 나눠준 주의의 비율, **합이 1**.
- `output[i]` = `weights[i]`라는 가중치로 V 의 n개 행을 **가중합**한 값 — "i번째 단어가 다른 단어들에게 요청한 정보(V)들을 관련도만큼 섞은 결과".

softmax를 다시 볼 이유: 차이를 증폭

$$p_2 = \frac{e^{g}}{1 + e^{g}} = \frac{1}{1 + e^{-g}} = \sigma(g)$$

softmax는 시그모이드 $\sigma(g)$와 동일

원점수 차이 g	가중치 (p_1, p_2)	해석
0.5	(0.378, 0.622)	아직 균형
2	(0.119, 0.881)	한쪽 우세
8	(0.0003, 0.9997)	사실상 하드 선택

$\sqrt{d_k}$로 나누는 이유: 분산의 정수학

- 내적 $\sum_{t=1}^{d_k} q_t k_t$ 는 **독립한 0평균 곱 d_k개의 합**.
- 각 항 $q_t k_t$ 의 분산은 1 (독립 0평균·분산 1 변수 두 개의 곱)이고, 독립 변수의 합은 **분산이 더해진다**.

$$\mathbb{E}[q \cdot k] = 0, \qquad \mathrm{Var}[q \cdot k] = d_k \quad \Longrightarrow \quad \text{표준편차} = \sqrt{d_k}$$

내적은 $\sqrt{d_k}$ 스케일로 자라난다

실제로 재보기: 무작위 내적 200쌍

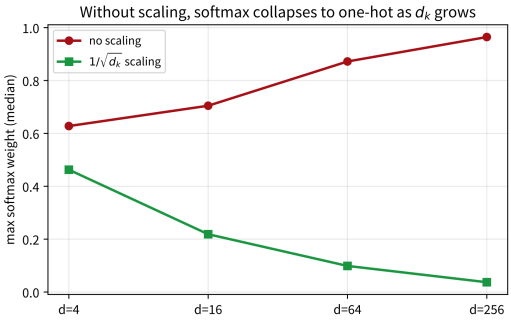
무작위 벡터(각 성분 $\mathcal{N}(0,1)$) 두 개를 200쌍 만들어 내적 계산 ($d = 64$):

	관측 범위
스케일링 **전**	$-19.1 \sim +22.3$
$\sqrt{64} = 8$으로 **후**	$-2.4 \sim +2.8$

- 최대치 22.3 $\approx 2.8\sigma$ — **기대와 일치**.
- 전의 $-19 \sim +22$면 softmax는 사실상 one-hot ($\sigma(22) \approx 1 - 2.7 \times 10^{-10}$).
- $\sqrt{64} = 8$으로 나누면 $-2.4 \sim +2.8$ — "분화된 가중치"를 정상적으로 내는 범위.

$\sqrt{d_k}$ 는 "마법 숫자"가 아니라 **내적의 표준편차를 정확히 1로 되돌리는 상수** — "무작위 기준선"에서 얼마나 벗어났는지를 표준편차 단위로 재게 한다.

$\sqrt{d_k}$는 내적 표준편차를 1로 환원하는 스케일



차원 d_k 가 커질 때, 스케일링 없이 d: 4→256이면 softmax **최대 가중치 0.63→0.96** (one-hot에 수렴); $1/\sqrt{d_k}$ 스케일링은 d와 무관하게 **0.46→0.04**로 낮게(분화되게) 유지 (500회 평균, 실습 실험 6).

온도(temperature) 관점

- d_k**로 나누면 신호 소실**
- **정확한 정규화 문제**

더 일반적인 형태: 온도 t

어텐션 원점수를 온도 t 로 나눠 softmax(s/t) 로 적용. t 크면 가중치가 **균등에 가까워지고**, t 작으면 **극단적**(한 단어 독주). $\sqrt{d_k}$ 스케일링은 초기 무작위 상태에서 원점수의 표준편차가 $\sqrt{d_k}$ 라는 사실을 이용해 $t = \sqrt{d_k}$ **라는 "정규 온도"**를 선택한 것. 실전에서 이 온도 관점으로 어텐션을 의도적으로 "분산"/"집중"하게 조절하기도 한다.

실습: 원점수 2개만 있는 예로 흐름 추적

$n = 2$, $d_k = 2$ 아주 작은 예. $Q = K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $V = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix}$.

1. **Step 1 — QK^T**: 항등행렬이므로 $QK^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. (1,1) 성분 $= 1 \times 1 + 0 \times 0 = 1$.
2. **Step 2 — $\sqrt{d_k} = \sqrt{2} \approx 1.414$ 로 나누기**: $\dfrac{QK^T}{\sqrt{2}} = \begin{pmatrix} 0.707 & 0 \\ 0 & 0.707 \end{pmatrix}$.
3. **Step 3 — 첫 행 $[0.707, 0]$ 에 softmax**: $e^{0.707} \approx 2.028$, $e^0 = 1$, 합 3.028.

$$p_1 = \frac{2.028}{3.028} \approx 0.670, \qquad p_2 = \frac{1}{3.028} \approx 0.330$$

   첫 단어는 자기 자신에 67%, 다른 단어에 33% — Q·K가 같은 벡터라 **자기 비교가 항상 한 번 더 유리한 구조**(아래 "자기 주입" 참고).
4. **Step 4 — 가중합**: $\text{out}_1 = 0.670 \times V_1 + 0.330 \times V_2 = (6.70, 3.30)$.

출력 $(6.70, 3.30)$ 을 V 의 원행들 $(10, 0), (0, 10)$ 과 비교하면 **첫 번째 V 에 67%의 "분량"이 더 섞여 있다**는 것이 직접 보인다 — "관련도가 높은 단어의 Value가 더 많이 기여한다"는 어텐션의 **핵심 행동**이 숫자로 구현된 모습. (이 예는 12.3절 연습문제 3의 설정과 정확히 같다.)

핵심 한 줄

"관련 단어의 내용을 관련도만큼 섞는다" — 이 문장이 $QK^T \to$ 스케일링 $\to$ softmax $\to V$ 가중합의 4단계를 통과하면서 숫자가 되는 과정이다.

어텐션 출력의 볼록성: 볼록 껍질 안에만 존재

- **출력은 V 원행들의 볼록 껍질 안에 존재**

실전에서 이어지는 두 가지

(1) **출력의 스케일이 입력 스케일의 범위 안에 갇혀 있다** — V 가 큰 값으로 학습되면 출력도 그 범위 안에서만 "미세 조정" (때로는 장점 — 안정성, 때로는 한계 — "V 가 표현할 수 없는 값"은 한 층으로 만들 수 없음). (2) **가중치가 극단(one-hot)이면 출력은 V 의 어떤 원행에, 균등이면 평균에** 가까워진다 — softmax는 "어떤 원행을 얼마나 섞을지"의 **비율**, 새로운 값이 아니라 섞임을 만드는 도구.

Self-Attention: 문장이 자기 자신을 본다

Self-Attention: 같은 문장 내 관련도 계산

구조를 한 줄로

각 단어의 “새로운 표현” = 같은 문장 안의 모든 단어(자기 자신 포함)의 Value 벡터를, “그 단어가 각각과 얼마나 관련 있는가”를 softmax로 나눠 **가중평균한 값**.

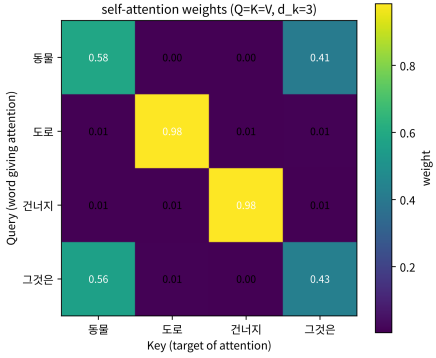
각 단어의 출력이 서로 독립이 아니다 — 앞 네 단어 예에서 “동물”의 출력에 “그것은”의 Value가 41.2% 섞여 있는 동시에, “그것은”의 출력에도 “동물”의 Value가 56.1% 섞인다. 두 단어는 서로의 표현을 서로에게 공급하면서 **동시에 업데이트** — RNN의 순차적 은닉 상태 업데이트와 **가장 근본적으로 다른 점** (12.1절의 병렬성 문제 바로 그 차이).

실습: 대명사 지시 관계를 시각화

(self-attention) 네 단어 [동물, 도로, 건너지, 그것은] 에 직접 설계한 임베딩으로 self-attention:

```
words = ["동물", "도로", "건너지", "그것은"]
E = [[3,0,0], [0,3,0], [0,0,3], [2.8,0.3,0]]
# 그것은이"" 동물과"" 유사하게설계됨
Q = K = V = E
output, weights = attention(Q, K, V, d_k=3)
for w, row in zip(words, weights):
    print(w, [round(x, 3) for x in row])
```

	동물	도로	건너지	그것은
동물	0.582	0.003	0.003	0.412
도로	0.005	0.980	0.005	0.009
건너지	0.005	0.005	0.984	0.005
그것은	**0.561**	0.007	0.004	0.427



"그것은" 행이 "동물" 열(0.561)과 자기 자신 열(0.427)에만 높은 값 — 대명사-선행사 관계가 숫자로 구현된 모습. 나머지 행은 대각선(자기 자신)에 집중.

왜 softmax가 이렇게 극단적인가 — 원점수에서

- $QK^T/\sqrt{3}$ **행: 동물(4.85)과 자신(4.578)이 크게 벌어짐**
- **softmax가 차이를 지수적으로 증폭하여 가중치 변환**

두 단계의 “스케일이 다르다”

softmax 이전: “동물”과 “도로”의 원점수 차이 **4.33**이라는 절대적 크기. softmax 이후: **56배의 가중치 비**라는 상대적 비율. — 이 스케일 차이가 “softmax가 차이를 증폭한다”는 관찰의 구체적 수치다.

이 예의 한계를 정확히 짚어두기

- d_k가 크면 스케일링 없이는 one-hot으로 붙는다

자주 하는 실수: QK^T 의 행/열 방향 혼동

확인법은 단순: QK^T 의 (i, j) 성분은 Q 의 i번째 행과 K^T 의 j번째 열, 즉 Q 의 i번째 행과 K **의 j번째 행**의 내적.

읽는 법

행 = “어떤 단어가 (Query로) 보고 있는가” (누가 주의를 주고 있나)
열 = “어떤 단어가 (Key로) 보고받고 있는가” (누가 주의의 대상인가)

네 단어 예: 행과 열의 의미

어텐션은 대칭적이지 않다

- “그것은 $\rightarrow$ 동물”의 가중치 **0.561**은 “동물 $\rightarrow$ 그것은”의 가중치 **0.412**와 같지 않다.
- **어텐션은 대칭적이지 않다.**

자주 하는 실수: d_k 를 “임베딩 차원”으로 오해

- d_k 는 **Key 벡터의 차원**, 즉 W_K 가 만들어낸 벡터의 길이.
- $d_{model} = 512$ 인데 $d_k = 64$ 라면, K 는 64차원 벡터이고 스케일 상수는 $\sqrt{64} = 8$.
- **“비교에 쓸 차원”(d_k)과 “전달할 내용의 차원”(d_v, Value의 차원)을 독립적으로 설계**할 수 있게 해준다.

확인 문제 4개(요약)

1. **(원점수 손계산)** $n = 3, d_k = 2$ 로 QK^T 를 구하고, 3번째 행을 $\sqrt{2}$ 스케일링 + softmax. 힌트: Q 의 3번째 행 $(1, 1)$ 과 K 의 세 행 $(1, 0), (1, 1), (0, 1)$ 의 내적 = 1, 2, 1 $\Rightarrow [0.707, 1.414, 0.707] \rightarrow\approx [0.248, 0.503, 0.248]$ — 가운데 K 에 가장 높은 가중치.
2. **(스케일링의 효과)** 네 단어 예($d_k = 3$)의 임베딩을 10배로 확대하면 QK^T 는 100배 $\Rightarrow \sqrt{d_k}$ 스케일링은 그대로이므로 softmax가 극단적으로 one-hot. “스케일링 상수는 초기 분산 기준이라, 임베딩 스케일이 변하면 다시 조정”해야 한다.
3. **(순서에 대한 대칭)** 임베딩 $[x_1, x_2]$ 와 순서 뒤집은 $[x_2, x_1]$ 에 self-attention 적용 — 각 단어의 출력(대표)은 같지만, W 행렬은 행/열 인덱스가 재배열된 **퍼뮤테이션-등변** 관계. “순서 정보의 결핍”이 12.3절 PE의 동기.
4. **(볼록성 검증 — 코드)** 각 출력 벡터가 V 의 4개 원행 최소/최대 성분 범위 안에 있는지 필요 조건으로 확인. (“볼록 껍질 안에 있으면 범위에 반드시 있지만, 범위에 있어도 볼록 껍질 안에 있지 않다”)

자주 묻는 질문 1: 문법적으로 "옳은" 대상에 집중하나

어텐션 가중치가 항상 "문법적으로 옳은" 대상에 집중 하나요?

- **항상 그렇지 않다.** 학습된 모델의 어텐션 헤드를 실제로 들여다보면, 어떤 헤드는 정확히 대명사-선행사 관계에 집중하지만, 다른 헤드는 그냥 **바로 이웃한 단어나 문장부호**에 집중하는 등 사람이 보기에 "덜 해석 가능한" 패턴을 학습하기도 한다 (Michel et al., 2019).
- 12.3절의 **멀티헤드 구조**가 바로 이런 "서로 다른 역할을 가진 여러 헤드"를 병렬로 두는 이유.

자주 묻는 질문 2·3: 변환이 없으면?

softmax 없이 내적값을 바로 가중치로 쓰면 안 되나요? 내적값은 **범위가 정해져 있지 않고**(음수 가능), 단어마다 **스케일이 달라져** "합이 1" 보장이 없으니 출력 스케일이 불안정. softmax는 Week 2.3 로지스틱회귀와 **동일한 역할** — 임의의 실수를 "**합이 1인, 항상 양수인** 분포"로 바꿔 안정적인 **보간(interpolation)**으로.

V 도 같은 임베딩에서 나오는데, 왜 서로 다른 W_V 가 필요한가? $V = X$ 로 못 하나?

- 가능하지만 **표현력이 제한**: $V = X$ 라면 출력은 "입력 임베딩들의 가중합"이므로 **한 층의 출력이 항상 입력 임베딩 공간 안에 갇혀** 있다.
- W_V 를 두면 "**비교에 쓰는 표현**(Q,K)"과 "**전달하는 내용**(V)"을 서로 독립적으로 학습 — 예: "그것은"의 Key는 "나를 가리킬 명사"에 높은 점수를 줘야 하고(비교용), Value는 "지쳐 있음" 같은 내용을 담아줘야 — 두 역할을 같은 벡터로 동시에 충당하기 어렵다.

참고: "$Q = K = V = X$"만으로도 어텐션 가중치 자체는 일단 계산된다 (앞 실습이 그 설정) — 가중치 패턴이 의미적으로 잘 나오는지는 W_Q, W_K, W_V 가 학습되었을 때의 문제.

자주 묻는 질문 4: softmax의 그래디언트

$p_i = e^{s_i} / \sum_j e^{s_j}$ 의 s_k 에 대한 미분:

$$\frac{\partial p_i}{\partial s_k} = p_i(\delta_{ik} - p_k) \qquad \text{(크로네커 델타 } \delta_{ik})$$

해석: “내 확률 p_i 가, 내 원점수 s_k 를 올리면 $p_i(1 - p_k)$ 만큼 증가하고, 다른 원점수 s_j $(j \neq k)$ 를 올리면 $p_i p_j$ 만큼 감소.”

중요한 관찰

p_i **가 1에 가까우면(one-hot)** $1 - p_k \approx 1 - p_i \approx 0$ 이 되어 그 방향의 **그래디언트가 사실상 0** — “이미 한 단어를 독점하면, 그 선택을 바꿀 학습 신호가 거의 안 온다.” 이게 “softmax가 one-hot으로 붙으면 학습이 멈춘다”는 앞 절 관찰의 **정확한 미분적 근거**.

이 미분 공식 자체는 Week 3 다중 분류 교차엔트로피 그래디언트 $(p - y)$ 와 정확히 같은 구조 — “softmax + 교차엔트로피”의 그래디언트가 선형으로 단순해지는 이유를 Week 3에서 본 바로 그 사실.

자주 묻는 질문 5: “자기 주입”은 왜 대각선이 높나

Q와 K가 같은 임베딩이면, $q_i \cdot k_i = \|q_i\|^2 \geq 0$ (자기 자신과의 내적 = **노름 제곱, 항상 0 이상**).

Cauchy-Schwarz 부등식 $q_i \cdot q_j \leq \|q_i\| \, \|q_j\|$ 에 따르면, 다른 단어와의 내적이 자기 자신과의 내적보다 클 수 있는 경우는 q_j 의 노름이 q_i 보다 크고 방향까지 비슷할 때뿐 — 임베딩 노름이 서로 비슷하면 **자기 자신과의 내적이 최대가 되는 구조**.

네 단어 예에서 “도로”가 자기 자신에 **98.0%**를 주는 것이 정확히 이 구조의 산출물 — “도로”와 다른 단어 (동물 $(3, 0, 0)$, 건너지 $(0, 0, 3)$)와의 내적이 0인 반면, “도로”와 자기 자신 $(0, 3, 0)$ 과의 내적이 $3^2 = 9$ 이어서 softmax에서 압도적.

이 “자기 주입”은 **단점이 아니다** — 각 단어의 표현에 자기 자신의 Value를 기본적으로 포함시켜 “이 단어의 기존 정보가 완전히 사라지지 않고” 다른 단어의 정보가 추가되는 형태”로 동작 → **학습 안정성에 긍정적**.

“자기 주입 + 잔차” 조합은 10.2절 ResNet(He et al., 2015)의 $y = F(x) + x$ 와 정신적으로 닮아 있다 — “출력 = 입력의 일부 + 새로운 정보” 패턴.

12.3 멀티헤드 어텐션과 포지셔널 인코딩

Multi-Head Attention: 관점을 병렬로

여러 헤드 병렬로 관련성 학습

- 여러 헤드의 결과를 **이어붙인(concatenate)** 뒤 다시 한 번 **선형변환**해서 최종 출력을 만든다.

Week 7.2 랜덤 포레스트(Breiman, 2001)와 같은 직관

"서로 다른 트리 여러 개의 다수결"로 단일 트리보다 안정적인 예측을 얻었던 것과 비슷. 다만 RF의 트리들은 **무작위성**만으로 다양해지는 반면, 멀티헤드의 각 헤드는 **역전파**를 통해 "서로 다른 역할을 맡는 것이 전체 손실을 줄이는 데 유리하다"는 방향으로 **스스로 분화**한다는 차이가 있다.

왜 헤드 하나가 부족할까 — “관점”의 문제

“크기”의 문제가 아니라 “관점”의 문제.

- 한 개의 어텐션 헤드는 단어 쌍마다 **점수(내적) 하나**를 만든다 — 그 하나의 숫자가 그 쌍의 모든 관계를 짊어져야 한다.
- 한 개의 점수는 이 둘을 **타협**해야 하지만, 두 개의 헤드는 각자 하나의 요인에 전부 집중할 수 있다 — 헤드가 두 개라 **“타협 대신 전담”**이 가능.

기하학적으로: 볼록 결합에서 직합으로

- 한 헤드는 각 단어에 대해 V 원행들의 **볼록 결합 하나**(볼록 껍질 안의 점 하나)를 만든다.
- 표현 가능한 영역이 **단일 볼록 결합에서 여러 부분공간의 볼록 결합의 직합(direct sum)으로 넓어진다**.

랜덤 포레스트 비유를 다시

단일 트리가 “한 번의 투표”라면 포레스트는 여러 투표의 병합. 헤드들도 마찬가지로 **“한 관점의 투표를 여러 개 붙여 놓는”** 것이다.

멀티헤드 공식

모델 차원 d_{model} 을 h개의 헤드에 나눠, i번째 헤드는 $d_k = d_{model}/h$ 차원의 Q, K, V 위에서 12.2절의 어텐션을 계산:

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,\dots,\text{head}_h)\,W_O, \qquad \text{head}_i = \text{Attention}\Big(QW_i^Q,\ K$$

d_{model} 을 h로 나눈 이유

헤드를 “크게” 만드는 것이 아니라 “다” 만드는 것.

- $d_k = d_{model}/h$ 로 나눠야 **파라미터 수가 단일 헤드와 (거의) 같다**:

$$\underbrace{h \times 3 \times d_{model} \times \frac{d_{model}}{h}}_{\text{h 헤드의 QKV 합계}} = 3\, d_{model}^2 = \underbrace{3 \times d_{model} \times d_{model}}_{\text{단일 헤드 QKV}}$$

$d_{model} = 512,\ h = 8$ 으로 대면: $8 \times 3 \times 512 \times 64 = 786{,}432 = 3 \times 512 \times 512$ — **정확히 같다.**
(여기에 W_O(512×512 = 262,144)만 더해진다.)

설계의 요지

파라미터 비용을 거의 그대로 유지한 채 독립적 관점의 수를 h배로 사는 것 — 더 많은 파라미터가 아니라 같은 파라미터를 **재배분**하는 설계. (12.2절 “d_k 는 임베딩 차원이 아니라 헤드당 Key 차원”이라는 사실의 설계적 귀결.)

손으로 한 번: 두 헤드(자기 / 이웃)

- **헤드 A** (자기): $W_Q = W_K = W_V = I$ — 12.2절의 “자기 주입” 헤드.
- **헤드 B:** W_K**만 차원 교환 행렬**

헤드	W_K	어텐션 가중치 (행=Query)	읽기
A (자기)	I	(0.670, 0.330) / (0.330, 0.670)	각 단어가 **자기 자신**에 67%
B (이웃)	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	(0.330, 0.670) / (0.670, 0.330)	각 단어가 **다른 단어**에 67%

- 헤드 A의 단어 1 출력 = $[0.670, 0.330]$
- 헤드 B의 단어 1 출력 = $[0.330, 0.670]$

이어붙임: 하나의 벡터 안에 두 관점

$[0.670,\ 0.330,\ 0.330,\ 0.670]$, 단어 2: $[0.330,\ 0.670,\ 0.670,\ 0.330]$

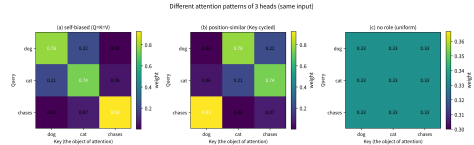
멀티헤드는 두 사실을 나란히 넣었다

$$[0.670, 0.330] + [0.330, 0.670] = [1.0, 1.0] \quad (\text{단어 2도 } [1.0, 1.0])$$

이어붙임은 소거를 막고 자유도를 남긴다

세 단어 [개, 고양이, 쫓는다]: 서로 다른 3개 헤드

- **(a) 자기-우위 헤드**: $Q = K = V$ 로 self-attention — 12.2절 “자기 주입” 효과 (자기 자신과의 내적 $\|q_i\|^2$ 가 최대)로 **대각선 우세** (“개” 0.78, “고양이” 0.74, “쫓는다” 0.93).
- **(b) 위치-유사 헤드**: Key만 1위치 순환 이동 ($K_i = E_{i+1}$) — 각 단어는 **다음 단어**의 Key와 가장 잘 맞아 (개→고양이 0.78, 고양이→쫓는다 0.74, 쫓는다→개 0.93) — 위상(순서) 정보가 가중치에 새겨진다.
- **(c) 역할 없음**: 모든 Key를 $(1, 1)$ 으로 — 모든 가중치가 **0.333 균일**, “아직 아무 역할도 분화하지 못한” 헤드.



3단어에 대해 패턴이 서로 다른 3개 헤드. 학습된 모델에는 (a)(b) 같은 서로 다른 패턴의 헤드들이 실제로 공존한다.

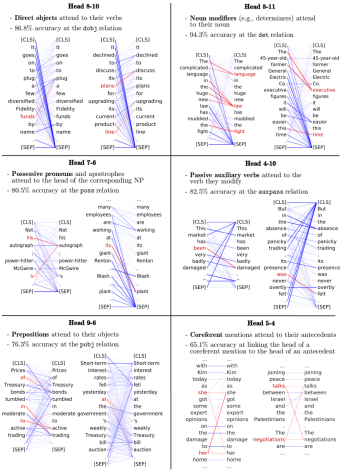
역사: 헤드 역할 분화는 관측된 사실인가

네 — "인터프리테이빌리티" 연구로 실제로 확인:

- Clark·Khandelwal·Levy·Manning ("What does BERT look at?", 2019)은 BERT(Devlin et al., 2018)의 헤드를 하나씩 분석:
  1. **바로 앞 단어만** 보는 "위치 헤드"
  2. **같은 품사**의 단어끼리 집중하는 헤드
  3. **문법적 통사**(주어-구문)를 추적하는 헤드

  가 **분명히 존재**함을 보였다.
- **부수적 결과**: 헤드의 50~60%를 임의로 제거해도 성능 저하 **매우 작다** — 역할 분화는 실제로 있으면서도 **여분(redundancy)**이 크다 (이 때문에 **헤드 프루닝** (Michel et al., 2019)이 추론 속도 기법으로 쓰임).
- **주의**: 어떤 헤드가 어떤 역할을 맡는가는 학습 run마다 **다르다**.

2017년 ablation: 헤드를 8→**1개**로 줄이면 BLEU가 **약 1점** 감소 (EN-DE 24.16→23.48, EN-FR 38.19→37.19) — "관점 하나만"이면 눈에 띄게 손해; 8개 이상은 **감소수익**. 헤드당 차원이 너무 얇아지면($d_k = d_{model}/h$, 예: $h = 32$이면 $d_k = 16$) 이득이 사라진다.



"What Does BERT Look At?" Figure 5 — 특정 헤드들은 인접 토큰, 품사 같은 토큰, 통사 구조적으로 연결된 토큰(주어-동사, 대명사-선행사)에 집중.

현재 대형 LLM(GPT(Radford et al., 2018), LLaMA(Touvron et al., 2023) 등)

헤드를 **수십~100개 이상** 쓰지만 d_{model} 이 4096~8192로 비례로 커진 덕분이며, **헤드당 차원은 여전히 64~128 부근 — 2017년 설계가 그대로 살아 있는 값**.

Positional Encoding: 순서 정보는 어떻게 넣나

어텐션은 순서를 구분하지 않는다

- 두 문장의 어텐션 가중치 행렬:

$$W_{\mathrm{B}} = P\, W_{\mathrm{A}}\, P^{T}$$

(완전히 같은 관계 — P 는 1·2행을 바꾼 **퍼뮤테이션 행렬**).

permutation-equivariant(퍼뮤테이션-등변)

자기 어텐션 전체 연산이 **입력을 재배열하면 출력도 같은 방식으로 재배열**. 순서 정보를 어딘가에 덧붙여 주지 않는 한 “개가 고양이를 쫓는다”와 “고양이가 개를 쫓는다”를 구분하는 것은 **수학적으로 불가능**.

PE의 수식: 시계 쌍

위치 t 의 d_{model} 차원 벡터를, **짝수 차원에 사인 · 홀수 차원에 코사인**을 두는 식:

$$PE_{(t,\,2i)} = \sin\left(\frac{t}{10000^{2i/d_{model}}}\right), \qquad PE_{(t,\,2i+1)} = \cos\left(\frac{t}{10000^{2i/d_{model}}}\right)$$

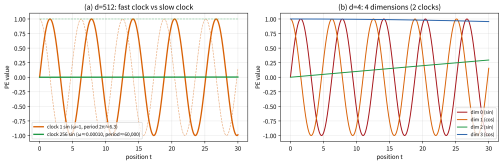
- 가장 작은 $d_{model} = 4$ 로 값을 써보면:

위치 t	$PE(t)$ (d=4)
0	(0.000, 1.000, 0.000, 1.000)
1	(0.841, 0.540, 0.010, 1.000)
2	(0.909, −0.416, 0.020, 0.9998)
10	(−0.544, −0.839, 0.0998, 0.995)

1위치 이동 = 1라디안 회전

256개의 시계 = 거의 유일한 512차원 패턴

- 256개의 시계가 모두를 합치면, **수만 단어 범위 안의 서로 다른 위치마다 거의 유일한 512차원 패턴**이 생긴다.
- 2017년 모델이 **512 길이의 문장**을 다룰 수 있었던 이유이기도 하다.
- 빠른 시계(주기 ≈ 6.3)는 **이웃 위치**를 구분, 느린 시계(주기 $\approx 60{,}000$)는 **문장 전체의 "대략 어디쯤"**을 표시.



(a) d=512의 1번(가장 빠른) 시계 vs 256번 (가장 느린) 시계의 사인/코사인 곡선. (b) d=4의 4개 차원 (2개 시계)을 위치 $t = 0 \sim 30$ 에 따라 그린 값 — 서로 다른 주기의 파동이 겹쳐 **각 위치마다 고유한 패턴**.

사인/코사인의 수학: 회전 성질

핵심 성질: 위치 k 만큼 앞서는 것은, 시계 바늘을 각도 k 만큼 돌

즉, ω 에만 의존하는 고정된 2×2 회전행렬 $R(\omega)$ 가 존재해

$$PE_{(t+k)} = R(\omega)^k \, PE_{(t)}$$

가 성립(중학 수학의 "각도 k 회전 = 사인/코사인 가산 정리의 행렬 버전"). 2시계로 묶어 $R = \text{blockdiag}(R(\omega_1), R(\omega_2))$ 로 쓰면, k**개의 위치 이동은 입력 벡터에 선형 변환 R^k 를 적용한 것과 정확히 같**다.

그것이 왜 좋은가 (원논문의 표현)

"각 위치 t 의 PE는 상대 위치 k 에 대한 **선형 함수**로 표현된다" ($PE_{(t+k)}$ 의 각 성분이 $PE_{(t)}$ 의 성분들의 선형 결합, 결합 계수는 k 에만 의존). 즉 **어텐션의 선형 도구**(Q·K 내적, W_O 선형변환)만으로 "**그 단어가 나보다** k**칸 뒤**"라는 상대 관계를 읽을 수 있다 — "절대 위치 7"이 아니라 **"그 단어와의 거리 3"**이 학습 대상. **차분 성질** $PE_{(t+k)} - PE_{(t)} = (R^k - I)\, PE_{(t)}$ 는 절대 위치 t 에 무관하고 k 만 담는 패턴을 만듦 (실습 실험 5, $t = 3, k = 2$ 수치 확인).

왜 “더하기”인가 — 그리고 왜 고정된 함수인가

1. **학습하지 않고 고정 함수를 쓰는 이유**:
   - **학습된 위치 임베딩**: “학습 때 본 최대 위치”가 **정의의 경계**.
   - $\sin(t/10000^{i})$: **아무** t **에도 정의된다**.

자주 하는 실수: PE를 빼먹고 어텐션만 쓰기

PE 없이 어텐션은 순서 정보 상실

정량화

PE를 넣자 같은 문장에서 “쫓는다”의 가중치 행이 **(0.187, 0.045, 0.768)** → **(0.013, 0.008, 0.979)** 로 달라진다 — **같은 단어라도 위치에 따라 어텐션이 달라진다**는 것, 이것이 PE가 실제로 하는 일 (실습 노트북 실험 4).

보너스: Masked Attention — 디코더는 왜 “왼쪽만” 보는가

- 디코더가 t번째 단어를 만드는 순간, $(t+1)$번째 이후(**“미래”**)의 정보를 보아서는 안 된다.
- **마스크: 상삼각에 $-\infty$ 추가**

causal(인과) 마스크의 효과

학습은 모든 위치를 병렬로 계산하되 각 위치가 왼쪽만 보는 것이 가능해진다 — **“학습의 병렬성”과 “생성의 순차성”을 하나의 마스크가 통일**. Week 17 LLM의 **자기회귀(autoregressive) 생성**은 정확히 이 메커니즘 위에 있다(디코더의 self-attention이 “masked/causal attention”이라 불리는 이유).

3단어 예: causal mask 전/후

위치	마스크 없은 가중치 행	마스크 있(causal) 가중치 행
1 (개)	(0.284, 0.140, 0.576)	**(1.000, 0, 0)**
2 (고양이)	(0.248, 0.503, 0.248)	(0.330, 0.670, **0**)
3 (쫓는다)	(0.187, 0.045, 0.768)	(0.187, 0.045, 0.768)

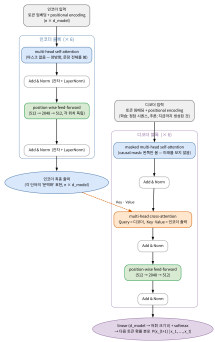
- **중간 위치 2에서 가중치 재분배**

train/test 일치

"softmax 이전에 $-\infty$ 를 더하면 t번째 위치가 $\leq t$ 위치만 본다" — 이 점이 **"학습 때의 t번째 출력"과 "실제 생성 때의 t번째 토큰"이 동일한 정보에 의존하게** 만들어 train/test 일치를 보장 (힌트: 학습 때 미래를 보게 하면, 추론 때 미래가 없는 상황이 분포적으로 달라져 모델이 학습 시 익숙하지 않은 상태에 처한다 — 과적합/분포 불일치 관점).

전체 트랜스포머 블록: 조각 조립

1. **multi-head self-attention** (+ **잔차 연결**, layer norm(Ba et al., 2016))
2. **position-wise feed-forward network**: 각 위치의 벡터를 독립적으로 2층 MLP ($d_{model} \rightarrow 4\,d_{model} \rightarrow d_{model}$)에 통과 (+ 잔차 연결, layer norm)



인코더/디코더 블록 구조

왜 attention 뒤에 FFN이 필요한가

- 섞은 결과에 **비선형 변환**을 적용하는 역할이 **FFN의 몫**.

분담

attention = "**어떤 정보를, 어떤 단어끼리, 얼마나 섞을지**", 그 뒤의 **FFN** = "**섞인 정보를 어떻게 처리할지**" — 9장 이후 반복된 **"선형 + 비선형"** 논리의 또 다른 실체.

참고: 파라미터 스케일의 감각

base 모델($d_{model} = 512$, 6 블록)에서 한 블록의 파라미터 대부분은 attention(QKV+W_O 합계 약 **39만**)보다 **FFN**이 차지 — $512 \to 2048 \to 512 = 2 \times 512 \times 2048 \approx$ **210만**. "**트랜스포머의 지식이 어디에 있는지**"를 찾는 연구 (FFN을 "키-값 메모리"로 본 해석 — Geva et al., 2020)에서 **사실상의 대부분이 FFN의 가중치에 담겨 있다**는 관찰이 이 비율과 맞닿아 있다.

RNN 대비 이점

Self-attention은 병렬 계산으로 GPU 활용 극대화

관점	RNN/LSTM (Ch.11)	Self-Attention (Ch.12)
1번째 $\rightarrow$ 100번째 단어 경로	99 단계(순차 언폴딩)	**1 단계**($Q \cdot K$ 직접)
병렬성	없음(순차 필수)	**모든 쌍 동시**
긴 거리 의존	BPTT로 그래디언트 소실	**구조적으로 없음**(경로 항상 1)
계산 복잡도(길이 n에 대해)	$O(n \cdot d^2)$ (선형)	$O(n^2 \cdot d)$ (**제곱**)
순서 정보	암묵적(순차 구조 자체가)	명시적 (PE를 입력에 추가)

마지막 행이 앞 네 행의 “대가(trade-off)”

- 병렬성과 경로 길이 이점을 사는 대가:
  1. **제곱 계산량** ($O(n^2)$),
  2. “**순서를 스스로 인코딩**해야 하는” PE라는 **명시적 장치**.
- 두 구조는 **같은 “순서” 정보를 다를 곳에 둔 것**.

자주 묻는 질문 1: $O(n^2)$ 가 실전에서 얼마나

문장 1,000단어면 $1000^2 = 100$만 쌍, 10,000단어면 1억 쌍 — **길이가 10배 늘면 계산량은 100배**. 이 때문에 아주 긴 문서(책 한 권 분량)를 다루는 LLM은 이 제곱 비용을 줄이는 **다양한 근사 기법**(희소 어텐션 — Child et al., 2019; 슬라이딩 윈도우 등)을 함께 쓴다.

FAQ 3개: PE와 헤드, RNN 메모리

- PE는 고정, PE를 쓰는 가중치가 학습
- RNN 은닉 상태와 어텐션은 다른 두 장치

12주차 정리

1. **12.1 — 어텐션은 RNN의 패치에서 시작** 압축 병목을 우회해 “원문 전체를 다시 보고 적절한 비율로 쓴다”. Q·K·V는 같은 입력에서 서로 다른 세 변환 — W_Q, W_K, W_V 가 **학습으로 결정**되며, 이 세 행렬의 존재가 어텐션의 핵심.
2. **12.2 — 4단계와 $\sqrt{d_k}$** $QK^T \to \sqrt{d_k}$ 스케일 $\to$ softmax $\to V$ 가중합. softmax는 원점수 차이를 지수적으로 증폭하고, $\sqrt{d_k}$ 는 **무작위 내적의 표준편차** (차원 d_k)를 1로 환원하는 상수 — one-hot으로 붙는 softmax를 막는 안전장치.
3. **12.3 — 관점 h배 + 순서 명시적 인코딩** 멀티헤드는 **파라미터를 재배분**해 독립 관점의 수를 h배로 산다. PE는 시계(사인/코사인)의 회전 성질로 상대 위치를 선형 도구로 읽게 해준다. causal mask가 **학습의 병렬성과 생성의 순차성**을 하나로 통일.

다음 주차 — Week 13. 그래프 표현학습

노드와 엣지로 표현되는 데이터를 벡터로: Node2Vec·PageRank. “어텐션이 단어 쌍의 관련성을 직접 계산”한다는 아이디어가 “그래프의 이웃 구조”로 이동한다.